

5.1 Project Overview

This project is a Category 1 multi-process application that uses pipes to implement a **concurrent file organization system**. The program scans the user-specified directory (relative to the project file's location), distributes file-processing tasks across at least 4 worker processes, and organizes files into structured subdirectories based on the information encoded in the filenames.

The parent process is responsible for orchestrating the system. It reads the user's given directory input, filters through the files to weed out the invalid ones using **check_file_name (line 54 in filesort.c and line 212 in implementation.c)**, and distributes these files as jobs to the worker process in a round robin style. A file is considered valid if it contains at least one underscore, exactly one period, and no white space characters. This ensures that filenames can be parsed consistently across the workers.

Each worker process receives jobs through a pipe (**job_pipe**), processes the files independently, and returns the result through a different pipe (**result_pipe**). In detail, the workers:

- Clean up the filenames using **clean_filename — line 105 in filesort.c and line 45 in implementation.c**
- Extract the directory structure that the user wants to store the file in using **create_target_path — line 106 in filesort.c and line 60 in implementation.c**
- Categorize files using **categorize_file — line 108 in filesort.c and line 91 in implementation.c**
- Compute the file statistics based on the category of the file, such as line count (**count_lines — line 113 in filesort.c and line 121 in implementation.c**), word count (**count_words — line 114 in filesort.c and line 148 in implementation.c**), and the size of the file (**count_size — line 120 in filesort.c and line 174 in implementation.c**)
- Move the given files into the new or existing directories using (**create_go_directory — line 122 in filesort.c and line 241 in implementation.c**)
 - If the folder already exists, the client would move into that directory; otherwise, it would create it.

The program demonstrates the required concurrency by running multiple workers in parallel, using the worker pool method, and coordinating them via pipes. The parent collects results asynchronously using **select() (line 245 in filesort.c)**, allowing it to handle multiple workers without blocking the others if an error occurs in one.

The program's input is a directory path, provided interactively by the user at the beginning of the program. The output consists of:

- Real-time logs indicating when the parent receives the processed data from a worker
- A final summary printed by **print_summary (line 311 in filesort.c and line 318 in implementation.c)** that includes the original filename, the clean filename, the target directory where the file has been moved, and the statistics, such as the number of files processed, the categories that each file belongs to, and the total number of lines and words in plain text files.

The program's implementation demonstrates process creation via **fork()**, interprocess communication via **pipes, reading and writing**, and handling concurrent results using **select()**.

5.2 Build Instructions

To build the project run:

make

This compiles all the **.c** files and produces the executable ones.

To run the program, run:

./a3

Where the program will prompt the user to:

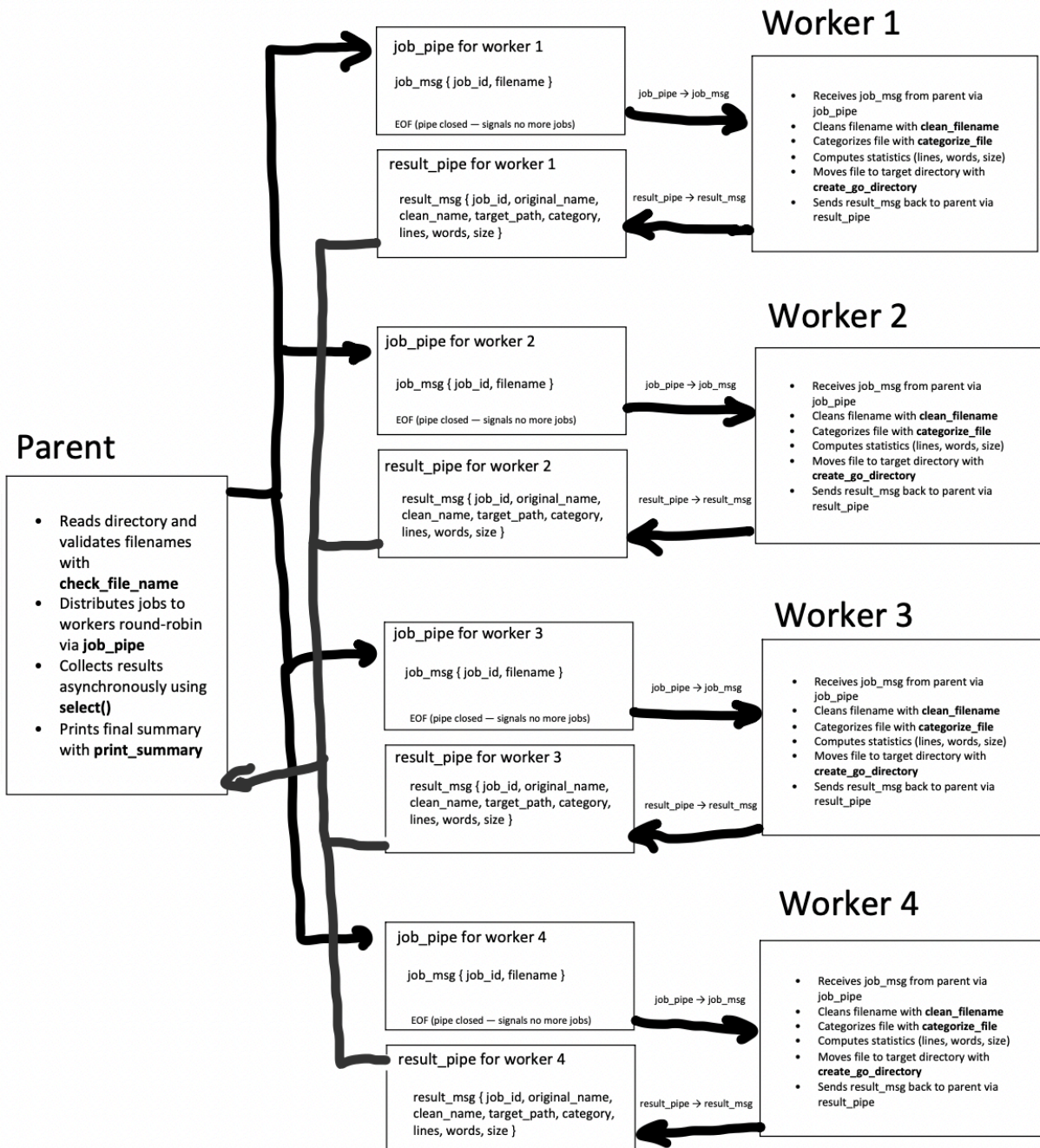
Enter the directory to organize:

The user must enter a valid directory path relative to the project's location on their computer.

The program includes a mock folder containing many files that simulate how it would run. In that case, enter:

messy_folder

5.3 Architecture Diagram



5.4 Communication Protocol

Field	Description
Direction	Parent → Worker
Encoding	A fixed-size binary struct written in a single write() call: <pre>typedef struct { int job_id; char filename[MAX_CHAR]; } job_msg;</pre> Where sizeof(job_msg) = sizeof(int) + MAX_CHAR bytes
Semantics and Invariants	Instructs the worker to process the specific new file. The id corresponds to the worker's index (i % WORKER_COUNT) The worker reads the struct, extracts the filename, and performs all the required processing steps. The workers read the data as: <code>read(job_pipe[i][0], &job, sizeof(job_msg))</code> (line 103 in filesort.c) Workers must read exactly <code>sizeof(job_msg)</code> bytes before processing. Filenames are guaranteed to be valid (checked by check_file_name — line 54 in filesort.c and line 212 in implementation.c in parent)
Error handling	If write() fails, the parent logs that it is skipping the specific file because it failed to send it to the worker, and continues the process. If read() returns 0, the worker interprets it as EOF and closes that worker.

Field	Description
-------	-------------

Direction	Worker → Parent
Encoding	A fixed-size binary struct written in a single write() call: <pre>typedef struct { int job_id; char original_name[MAX_CHAR]; char clean_name[MAX_CHAR]; char target_path[MAX_CHAR]; char category[MAX_CHAR]; int lines; int words; long size; } result_msg;</pre> Where sizeof(result_msg) = 5 * MAX_CHAR + 2 * sizeof(int) + sizeof(long) bytes
Semantics and Invariants	This message contains the results of processing the file, including the metadata and statistics. The parent reads and stores this information to later display it in the summary. - job_id is from the original job_msg - original_name contains the original file name - clean_name stores the new, cleaned-up file name - target_path contains the directory path of where the file has been moved - lines contains the number of lines in the file if the category is a plaintext/document - words contains the number of words in the file if the category is a plaintext/document - size is the byte size of the file The parent reads the data as: <code>read(fd, &result, sizeof(result_msg))</code> (line 267 in filesort.c) Parent reads exactly one full struct at a time. Results correspond to previously assigned jobs.
Error handling	If read() returns 0, the parent closes the pipe and marks the worker as finished. If write() fails in the worker, it logs to the user that the worker is skipping that file since there was an error with sending it back to the parent, frees the previously allocated memory, and continues to the next worker.

5.5 Concurrency Model

This program is a Category 1 multi-process concurrency model with a fixed worker pool.

At first, the program creates the necessary pipes so all workers can communicate back and forth with the parent. The parent then creates the pool of workers with **fork()** in **main** before distributing the work to each one. All workers immediately close the necessary file descriptors for each pipe (workers are only eligible to read from the **job_pipe** and write to the **result_pipe**), so they are ready to receive jobs as soon as the parent writes to that pipe.

The files are distributed in a round robin fashion:

int worker = i % WORKER_COUNT; (line 206 in filesort.c)

This ensures a nearly equal workload among all workers, so they are not overworked or slowed if the data gets too large. All workers run simultaneously and process the jobs independently of one another.

The parent uses **select()** to monitor all the **result_pipe[i][0]** (line 245 in filesort.c) file descriptors to avoid blocking on any single worker and to process the results as soon as they become available. This ensures that there is no huge flow of data back into the parent at the same time. Once the parents detect a ready response from the worker (line 271 in filesort.c), it reads the returned response and adds the ready data into respective arrays to later print them out in the summary to the user.

After sending all of the jobs, the parents close all of the job pipes. Workers detect EOF and exit elegantly. The parent then calls **wait()** (line 298 in filesort.c) to collect all the child processes and prevent zombies.

5.6 Error Handling and Robustness

1. Invalid or inaccessible directory

One bad behaviour that could occur is the user entering a directory that doesn't exist or that they don't have permission to open. This behaviour is checked in **filesort.c** (line 36 in **filesort.c**). If the **opendir** returns NULL, the program informs the user that it's unable to open

the directory (through **stderr**) and exits cleanly after freeing the previously allocated memory for **valid_files**. The directory is never iterated, and no workers are spawned.

2. The worker fails to write the result back to the parent

A second bad behaviour is if **write()** on **result_pip[i][1]** fails and the result never reaches the parent. This behaviour is handled in the workers' **while loop (line 104 in filesort.c)**. Instead of exiting immediately, the worker logs the error with **fprintf(stderr, “”)** (**line 132 in filesort.c**), frees all allocated memory for that job (**clean_name, target_path, new_directory**), and uses **continue** to skip to the next job. The parent's **select** loop doesn't hang because it relies on pipes closing, not on receiving a specific number of results. The summary will simply reflect fewer files than were found.

3. No valid files found in the directory

The third bad behaviour occurs when the directory the user inputs exists but contains no files that match the naming convention (at least one underscore, exactly one period, and no whitespace characters). This behaviour is handled and checked in **filesort.c** after the **readdir** loop (**line 43 in filesort.c**). It checks if **valid_file_count (line 64 in filesort.c)** is 0, then the program prints a message to the user indicating that there are no valid files in the given directory, frees allocated memory, and returns cleanly without spawning any workers or pipes.

5.7 Team Contributions

This project was completed individually by me, Sofia Borodaenko. I was responsible for all aspects of the project, including:

- Designing the parent-worker architecture
- Implementing inter-process communication using pipes
- Developing file parsing and validation logic
- Implementing more worker-side processing (categorization, file movement, statistics)
- Designing and implementing the asynchronous **select()** based concurrency model to ensure no process would get blocked if one worker takes too long
- Documenting the report